SKILL WEEK – 9

Task 1 - To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>

int main()
{
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX] = "";
    char input[PATH_MAX];

    printf("Simple CD Shell\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        if (getcwd(current_dir, sizeof(current_dir)) == NULL)
        {
            perror("getcwd");
            continue;
        }

        printf("%s$ ", current_dir);
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strcmp(input, "cd") == 0)
        {
            char temp[PATH_MAX];

            if (getcwd(temp, sizeof(temp)) == NULL)
            {
                perror("getcwd");
                continue;
            }

            if (chdir(getenv("HOME")) == -1)
            {
```

```
            {
                perror("cd");
            }
            else
            {
                strcpy(previous_dir, temp);
            }

            continue;
        }

        if (strncmp(input, "cd ", 3) == 0)
        {
            char path[PATH_MAX];

            strcpy(path, input + 3);

            if (strlen(path) == 0)
            {
                printf("cd: missing directory\n");
                continue;
            }

            if (strcmp(path, "-") == 0)
            {
                if (strlen(previous_dir) == 0)
                {
                    printf("cd: OLDPWD not set\n");
                    continue;
                }

                char temp[PATH_MAX];

                if (getcwd(temp, sizeof(temp)) == NULL)
                {
                    perror("getcwd");
                    continue;
                }

                if (chdir(previous_dir) == -1)
                {
                    perror("cd");
                }
                else
                {
                    strcpy(previous_dir, temp);

                    if (getcwd(current_dir, sizeof(current_dir)) != NULL)
                    {
                        printf("%s\n", current_dir);
                    }
```

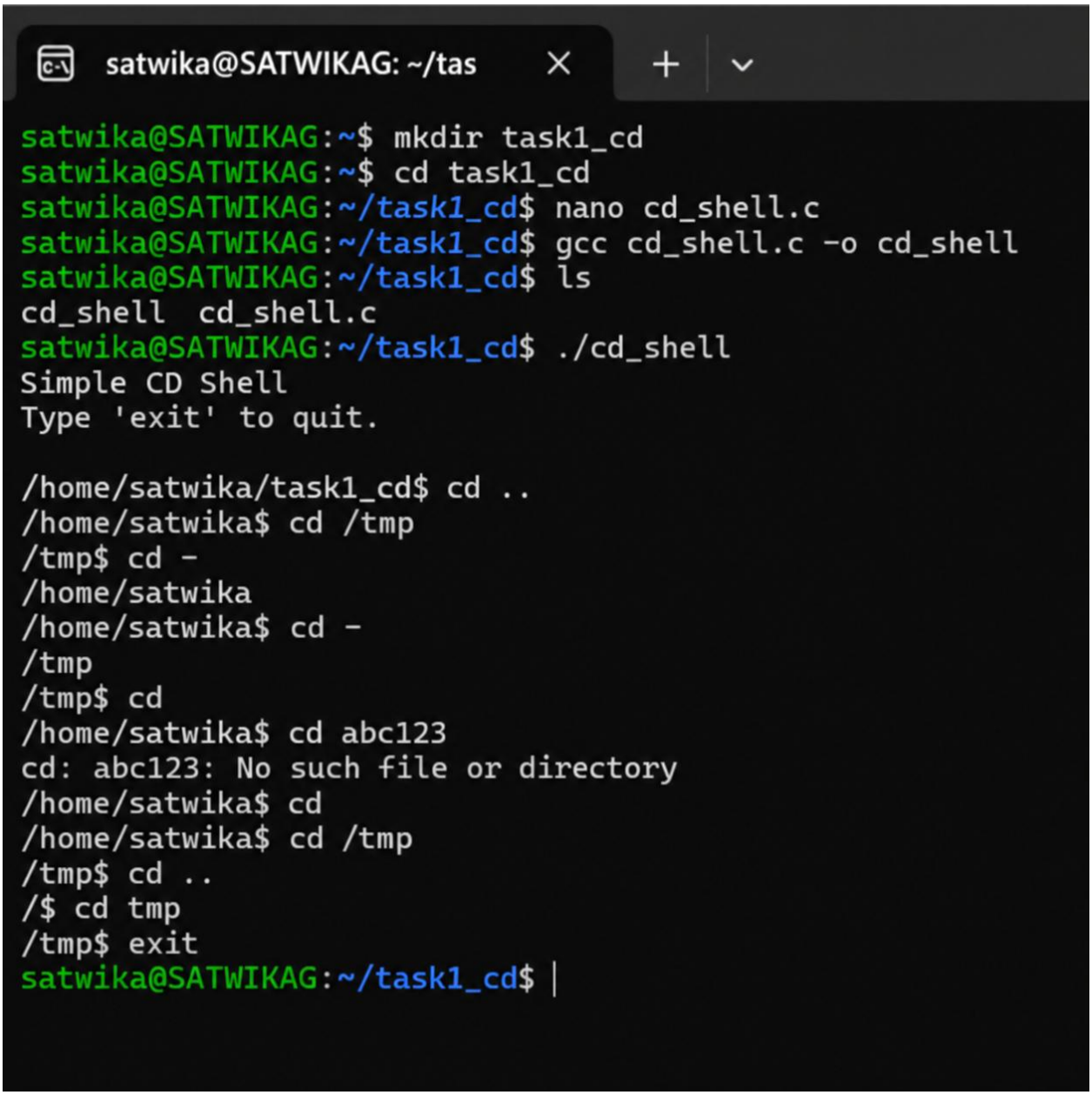
```
                }
            }

            continue;
        }

        char temp[PATH_MAX];

        if (getcwd(temp, sizeof(temp)) == NULL)
        {
            perror("getcwd");
            continue;
        }

        if (chdir(path) == -1)
        {
            printf("cd: %s: %s\n", path, strerror(errno));
        }
        else
        {
            strcpy(previous_dir, temp);
        }
```



Task 2 - To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100

/* Function declarations */
void builtin_pwd(void);
void builtin_cd(char *path);
void builtin_help(void);
void builtin_exit(void);

/* Structure for dispatch table */
struct builtin_command {
    char *name;
    void (*function)(void);
};

/* Dispatch table */
struct builtin_command commands[] = {
    {"pwd", builtin_pwd},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

int command_count = sizeof(commands) / sizeof(commands[0]);

/* pwd command */
void builtin_pwd(void)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

/* cd command */
void builtin_cd(char *path)
{
    if (path == NULL)
    {
        path = getenv("HOME");
    }

    if (chdir(path) != 0)
        perror("cd");
}
```

```
{
    printf("Available built-in commands:\n");
    printf("  pwd   - Show current directory\n");
    printf("  cd    - Change directory\n");
    printf("  help - Show available commands\n");
    printf("  exit - Exit the shell\n");
}

/* exit command */
void builtin_exit(void)
{
    printf("Exiting shell...\n");
    exit(0);
}

/* Find command in dispatch table */
int execute_builtin(char *command)
{
    for (int i = 0; i < command_count; i++)
    {
        if (strcmp(command, commands[i].name) == 0)
        {
            commands[i].function();
            return 1;
        }
    }

    return 0;
}

int main()
{
    char input[MAX_INPUT];

    printf("Simple Built-in Shell\n");
    printf("Type 'help' to see commands.\n\n");

    while (1)
    {
        printf("shell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        /* Handle cd separately because it needs an argument */
        if (strncmp(input, "cd", 2) == 0 &&
            (input[2] == ' ' || input[2] == '\0'))
        {
```

```
            char *path = input + 2;

            while (*path == ' ')
                path++;

            builtin_cd(*path ? path : NULL);
            continue;
        }

        /* Check dispatch table */
        if (execute_builtin(input))
            continue;

        /* Empty input */
        if (strlen(input) == 0)
            continue;

        printf("shell: command not found: %s\n", input);
    }

    return 0;
}
```

```
satwika@SATWIKAG:~/task1_cd$ mkdir task2_builtins
cd task2_builtins
satwika@SATWIKAG:~/task1_cd/task2_builtins$ nano builtin_shell.c
satwika@SATWIKAG:~/task1_cd/task2_builtins$ gcc builtin_shell.c -o builtin_shell
satwika@SATWIKAG:~/task1_cd/task2_builtins$ ls
builtin_shell  builtin_shell.c
satwika@SATWIKAG:~/task1_cd/task2_builtins$ ./builtin_shell
Simple Built-in Shell
Type 'help' to see commands.

shell$ pwd
/home/satwika/task1_cd/task2_builtins
shell$ help
Available built-in commands:
  pwd  - Show current directory
  cd   - Change directory
  help - Show available commands
  exit - Exit the shell
shell$ cd /tmp
shell$ pwd
/tmp
shell$ cd ..
shell$ pwd
/
shell$ hello
shell: command not found: hello
shell$ xyz
shell: command not found: xyz
shell$ struct builtin_command commands[] = {
    {"pwd",buillin_command not found: struct builtin_command commands[] = {
shell$ , builtin_pwd},
shell: command not found:     {"pwd", builtin_pwd},
shell$     {"help", builtin_help},
shell: command not found:         {"help", builtin_help},
shell$ {"exit", builtin_exit}
}:shell: command not found:     {"exit", builtin_exit}
shell$ pwd
shell: command not found: };pwd
shell$ help
Available built-in commands:
  pwd  - Show current directory
  cd   - Change directory
  help - Show available commands
  exit - Exit the shell
shell$ exit
```